claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-a5785411d62c14b6c.jsonl`
- SHA-256: `5ee25daccfeb946611d0ba3a7985ddb0be021869ceb610315ae344ae37c09b70`
- Source modified: `2026-06-10T16:18:33+00:00`
- Imported at: `2026-07-05T16:48:21+00:00`
- project: `wf_27df53c8-8e0`
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`

Transcript

1. user (2026-06-10T16:13:27.798Z)

You are auditing the ARCHITECTURE, VISION, ROADMAP, and product posture of gmail-attachments-to-gdrive (C:/Users/avidu/Projects/gmail-attachments-to-gdrive) ? an Apps Script project whose stated north star is 'turn a person's email into a high-fidelity, structured, queryable record of their life', positioned as the extraction/ingest front door that hands minimized signal to downstream processors (muneem for finance; future siblings for grocery/travel/coupons).

Read COMPLETELY: docs/VISION.md, docs/ARCHITECTURE.md, ROADMAP.md, README.md, PRIVACY.md, LICENSE, appsscript.json, package.json. Skim src/ to verify doc claims (e.g. the egress-guard test tests/auditability.test.ts, executeAs in appsscript.json, scopes requested).

Assess:
1. The phased plan (0 ingest model ? 1 GAS-only classify/extract/index ? 2 coverage/re-extraction ? 3 handoff to hosted services ? 4 surfaces/ops): is each phase realistic ON APPS SCRIPT? Specifically: classification + extraction + OCR of PDF attachments inside GAS quotas (6 min/run, 90 min/day triggers, ~20k Gmail reads/day, no native libs ? how would PDF OCR even run?); where does the index live (Sheets? size limits?); what does 'versioned handoff contract' mean concretely and does ANY artifact define it yet?
2. The auditability invariant (this repo is the only Gmail-touching code, self-deployable, no egress): is it enforceable as stated once Phase 3 (egress to hosted service) lands? Is 'logs to Cloud Logging' already egress? Does the multi-user web app weaken it?
3. Privacy/PII reasoning: bank statements ARE PII ? can the 'only non-PII signal leaves' rule and the muneem handoff coexist? Where would redaction of a bank statement PDF happen and is that feasible in GAS? Is PRIVACY.md consistent with VISION.md and with what the code does today?
4. Product posture: 'build for public, stay under 100 users (OAuth Testing mode)' ? check README's multi-user/publishing section; what's the real cost/process wall (gmail.readonly is a RESTRICTED scope ? annual CASA security assessment) and is the plan's 'decide later' stance coherent? What breaks at user 101?
5. Roadmap state: ROADMAP.md is mostly checked-done items plus a thin tail ? what SHOULD the next roadmap items be to serve the vision (the Phase-1 extractor framework? the handoff contract? SECURITY.md?). Is there any item tracking the grocery-receipts vertical the VISION names as first?
6. Architecture doc quality: decision log hygiene, anything contradicting muneem's docs (muneem has its own Gmail OAuth ingestion and a planned 'muneem fetch' ? VISION.md here says downstream never reads Gmail).

Return the structured report. The 'map' field must compactly state the vision, the phase plan, the auditability invariant, current implemented surface, and the publishing constraints as the docs state them. Structured output only.

2. user (2026-06-10T16:13:33.516Z)

```
1       # Vision ? Email as a smart way to organize your life
2
3       > Status: living document. The overall product direction. Architectural detail
4       > and the decision log live in [`ARCHITECTURE.md`](./ARCHITECTURE.md).
```

North star

Turn a person's email into a **high-fidelity, structured, queryable record of their life** ? receipts, bank statements, flights & travel, coupons, shopping ? built automatically, privately, and reliably. Email becomes the source of truth that organizes the rest.

What this repo is ? and is not

This repository (`gmail-attachments-to-gdrive`) is the **extraction / ingest layer**. It is the reliable, quota-aware "front door" that:

- watches a user's Gmail **in their own account**,
- archives raw artifacts (attachments) ? still valuable on its own,
- **classifies** messages and **extracts** a clean, normalized signal,
- **redacts/minimizes** that signal, and **hands it off** to downstream processors.

It is **not** the domain parser, the datastore, or the analytics engine. That logic is deliberately **isolated in separate downstream services**:

Concern	Owner
Gmail watching, archiving, classification, extraction, redaction, handoff	**this repo**
Bank-statement & financial parsing, indexing, encrypted storage, analytics	**Muneem** (`avidullu/muneem`)
Grocery / receipts, coupons, travel processing	future siblings of Muneem

The boundary is intentional: **extraction (this repo) is decoupled from processing (downstream).** This repo never imports domain-parsing logic; downstream services never read Gmail. They meet only at a versioned handoff contract.

Foundation first

Attachment archiving is the **hardened core**. Everything else is built on its proven strengths, which we preserve and extend, never bypass:

- per-user isolation (`executeAs: USER_ACCESSING`),
- a per-user lock and self-healing trigger fan-out,
- the hybrid `history.list` + date-window sync,
- "mark a message done only on clean completion,"
- pure-logic / GAS-shell separation with heavy unit testing,
- ruthless respect for Apps Script quotas and the 6-minute boundary.

Audience & rollout

- We **build for public** ? the long-term goal is a wide launch.
- **For now, stay under 100 users** (OAuth "Testing" mode) to harden the system, learn the value proposition, and make an *educated* call on whether to financially invest (the annual CASA security assessment, hosted infra) **once there is a substantial user base**.
- Until that call, we stay in Google's no-verification zone (see [`../README.md`](../README.md) ? Multi-user deployment / publishing).

Privacy north star

- **Minimize egress.** Only **non-PII** signal ever leaves the user's Google account, and only to **our own isolated, hosted service** ? **never third parties**.
- **Scrub + encrypt** as required, applied *before* anything leaves.
- Continuously **reduce** what leaves while maintaining product health.
- The in-account baseline needs **zero** external dependency; external

```
68        processing is a deliberate, **separately-consented** upgrade with a privacy
69        policy that names exactly what is sent, where, and why.
70
71      ### The auditability invariant
72
73      **This repo is the only code that ever touches a user's Gmail, and it stays
74      auditable and self-deployable by a power user at any time.** It is intended to be
75      open-sourced precisely so that a privacy-conscious user can read ? and run ? the
76      exact code that reads their mail. Muneem and every downstream service operate
77      only on minimized, non-PII signal and never hold the Gmail grant. This is a
78      binding constraint on all future work; the enforceable form (and a per-change
79      checklist) lives in [`ARCHITECTURE.md`](./ARCHITECTURE.md#auditability-invariant-
binding).
80
81      ## First verticals
82
83      1. **Grocery receipts** ? high volume, forgiving fidelity bar; proves the whole
84         ingest ? classify ? extract ? index pipeline cheaply.
85      2. **Bank statements** ? high-fidelity path; output feeds **Muneem**, proving the
86         redact ? handoff ? downstream-processing seam.
87
88      ## Success criteria (proposed)
89
90      - The user's index auto-populates with correct entities at **high precision**;
91        low-confidence items are **surfaced for review**, never silently dropped.
92      - Re-running an improved extractor over history **corrects past entities without
93        duplicates** and without re-copying blobs.
94      - Egress stays **minimal and non-PII**; the in-account baseline works with no
95        external service available.
96
97      ## Phased plan (high level)
98
99      Detailed status is tracked in [`../ROADMAP.md`](../ROADMAP.md); the technical
100     shape of each phase is in [`ARCHITECTURE.md`](./ARCHITECTURE.md).
101
102     0. **Ingest model** ? a rich, stable `EmailMessage`; the copier consumes it
103        (no behavior change).
104     1. **Classify + extract + index (GAS-only, in-account)** ? classifier, pluggable
105        extractors, normalized entities, a user-owned index. No egress.
106     2. **Coverage + re-extraction** ? more extractors, body/OCR handling, extraction
107        cursors & status, "re-extract for new rules," review queues.
108     3. **Handoff to hosted services** ? redact/minimize, then feed Muneem & siblings;
109        separate consent + privacy update; extraction/egress budgets.
110     4. **Surfaces & ops** ? dashboards, Calendar/Tasks automation, a richer review &
111        configuration UI, Shared Drives, schema migration.
112
```

3. user (2026-06-10T16:13:34.483Z)

```
1       # Architecture
2
3       > Status: living document. Captures the **target architecture** for the
4       > extraction pivot and the **decisions** behind it. Product framing is in
5       > [`VISION.md`](./VISION.md). This supersedes the original "just copy
6       > attachments" mental model; the copier survives as one extractor at the leaf.
7
8       ## Auditability invariant (binding)
9
10      **This repository is the _only_ component that reads or processes a user's
11      Gmail, and it must remain auditable and self-deployable by a power user at any
12      time.** This is the product's core privacy guarantee and a deliberate constraint
13      on every future decision (see ADR-008). It is why the project is built to be
14      open-sourced: *don't trust us ? read, and run, the code.*
15
16      Every change MUST preserve all of the following. A change that cannot is, by
```

definition, a decision to break the invariant and needs explicit owner sign-off.

1. **Sole Gmail surface.** Gmail is read/processed *only* here, in the user's own account (`executeAs: USER_ACCESSING`). No other component ? Muneem included ? ever holds the Gmail OAuth grant or sees raw mail. Muneem may *orchestrate* this app; it is never *itself* the Gmail reader.
2. **Egress == the audited boundary.** Nothing derived from a user's mail leaves their Google account except through the in-repo **REDACT/MINIMIZE** stage, carrying only minimized, non-PII signal, only to the user's own hosted service, never to third parties. The set of egress destinations is explicit and reviewable in source.
3. **Run-your-own-from-source.** A power user can build and deploy their own instance from the published source and obtain an equivalent app; the build is reproducible and the verification steps are documented ? "trust by running the code you read," not merely "read the code."
4. **No secrets in source.** Credentials exist only as deploy-time secrets; the repository contains nothing sensitive.
5. **Transparent over clever** at the read and egress/redaction boundaries ? prefer obviously-correct, reviewable code even at some cost to brevity.

PR review checklist (apply to every change):

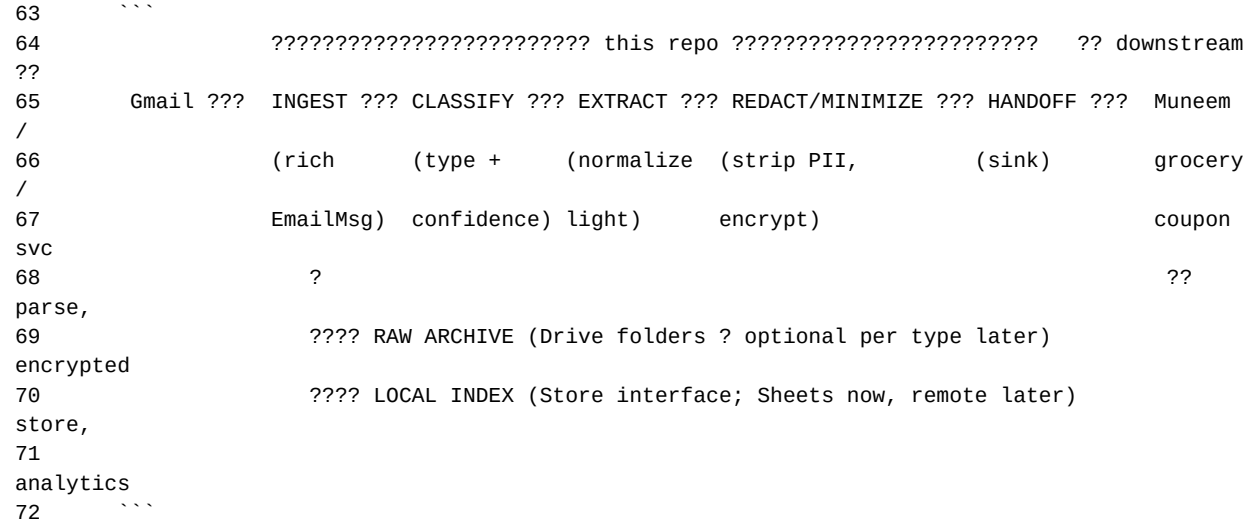
- [ ] Gmail access stays entirely within this repo, in-account?
- [ ] If any data leaves the account, does it pass through REDACT/MINIMIZE and go only to the user's own service (non-PII)?
- [ ] Build still reproducible and self-deployable from source?
- [ ] No new secrets, third-party endpoints, or network calls? (Default: none ? enforced by `tests/auditability.test.ts`.)

Principles (preserved from today's codebase)

1. **Shells around pure functions.** GAS-facing code is a thin shell; all logic is pure and unit-tested.
2. **Quota discipline.** Respect the 6-minute execution limit, the 90-min/day trigger budget, and per-user caps. Self-healing trigger fan-out; per-user lock; "mark done only on clean completion."
3. **Per-user isolation.** Runs as the accessing user; state is per-user.
4. **Versioning & provenance everywhere.** Every persisted shape carries a `schemaVersion`; every extracted entity carries where it came from and which extractor (and version) produced it.

The pipeline

This repo owns the left half; downstream services own the right half. They meet at the **handoff contract** only.



```
- **INGEST** builds a durable `EmailMessage` **once** per message (body, subject,
  labels, attachment metadata ? via the Gmail advanced service). Extraction
  re-runs over this record without re-touching Gmail or re-copying blobs.
- **CLASSIFY** assigns a type + confidence (sender/subject/body rules now; richer
  models downstream later).
- **EXTRACT** is a registry of pluggable, versioned extractors. The current
  attachment copier becomes the `RawArchive` extractor.
- **REDACT/MINIMIZE** enforces the privacy north star before any egress.
- **HANDOFF** routes a normalized payload to the right downstream **sink**.

## Data model

All pure shapes, versioned from day one.

```
EmailMessage {                          // INGEST output ? the stable substrate
  schemaVersion, id, threadId, historyId?,
  from, to[], cc[], subject, date, labels[],
  snippet, plainBody?, htmlBody?,
  attachments: [{ index, name, contentType, sizeBytes,
                  attachmentId/partId, sha256?, driveFileId? }],
}

ExtractedEntity {                       // EXTRACT output ? the queryable payload
  schemaVersion, entityKey,             // stable dedup key, e.g.
                                        //   receipt:{merchant}:{total}:{date}
  type, subtype, confidence,
  fields,                               // typed per kind (receipt, statement, ?)
  provenance: { sourceMsgIds[], attachmentIndex?,
                extractor, extractorVersion, extractedAt, model? },
  status: pending|extracted|low_confidence|needs_review|corrected,
  links: { driveFileId?, ? },
}

HandoffPayload {                        // REDACT/MINIMIZE output ? what leaves
  schemaVersion, entityKey, type, extractorVersion,
  nonPiiFields,                         // minimized, scrubbed signal only
  // encrypted in transit; PII stripped or tokenized
}
```

- **Entity keys** give content-stable dedup (re-mailed receipts; cross-source
  fusion of an email confirmation + a PDF ticket ? one entity).
- **Attachment `sha256`** enables content-addressable dedup of re-sent files.
- **Provenance + `extractorVersion`** make "this extraction is stale ? re-run"
  possible and auditable.

The extraction ? processing boundary

- This repo produces a normalized, classified, **redacted** `HandoffPayload` per
  detected document/entity and routes it to a downstream **sink**.
- Downstream services (Muneem for finance; siblings for groceries/coupons) own
  domain parsing, indexing, **encrypted storage**, and analytics.
- **Contract is owned jointly, defined from our side first.** The exact
  Muneem-facing schema and transport are **TBD and aligned with that repo**
  (this repo's GitHub tooling is scoped to itself, so the contract is specified
  here and reconciled with Muneem separately).
- Hard rule: this repo imports **no** domain-parsing logic; downstream reads
  no Gmail.

Storage abstraction (decision: extensible, not baked-in)

A single `Store` / `Sink` interface; code depends only on the interface.

| Impl | When | Properties |
```

```
| --- | --- | --- |
| `GoogleSheetsStore` (user-owned "LifeIndex" sheet) | v1 | in-account, queryable, zero infra, doubles as a first dashboard |
| `RemoteStore` (our hosted DB) / Muneem-API sink | later | the durable system of record |

`PropertiesService` stays only for small per-user **control state** (cursors, flags, counters) ? never the entity index.

## Privacy & egress controls

- **Egress only to our own isolated, hosted service. No third parties.**
- A **Redactor/Minimizer** stage runs before any egress: strip/tokenize PII, keep only non-PII signal, encrypt in transit (encrypted at rest downstream).
- North star: continuously **shrink** what leaves while keeping the product healthy.
- The in-account tier (Phases 0?2) has **no egress** and needs no privacy change. The external tier (Phase 3) is **flag-gated + separately consented**, and updates the privacy policy to name exactly what is sent and where.
- **Cascade to be aware of:** sending restricted-scope Gmail data outside Google is what makes Google's **CASA security assessment mandatory for a public launch**. Deferred ? consistent with "stay <100 until the user base justifies the investment."
- **Open tension to resolve (Phase 3 design):** bank-statement and receipt signal (merchant, amounts, account identifiers) is often itself sensitive. "Non-PII only" needs a concrete per-vertical definition of what is sent vs. tokenized vs. kept fully in-account.

## State & cursors

- Keep the hybrid `history.list` + date-window skeleton; **extend, don't duplicate.**
- Add an **extraction-aware cursor** ("fully extracted up to X with extractor-set V") distinct from the copy cursor, plus per-message extraction status ? so a "re-extract window for new rules" reprocesses without re-copying (content hash) or duplicating entities (entity keys).

## Quota, cost & pacing

- **Separate budgets:** raw-copy cap (existing) vs. extraction cap vs. egress/handoff cap.
- Per-item time budgets, **prioritization** (recent + high-confidence/trusted senders first), and **deferral** of heavy items.
- Heavy ML/PDF work happens **downstream**, not in Apps Script ? GAS stays the lightweight, reliable watcher/archiver/handoff.

## Observability

- Per-**classifier** and per-**extractor** success / failure / ambiguity rates ? not just "files added."
- Surfaced **review queues:** "matched no extractor" and "low confidence."
- Structured logs carrying `messageId`; extraction & handoff metrics.

## Platform spectrum

1. **Pure-GAS conservative extractors** (regex, known formats `.ics`/`.csv`, native Drive OCR via Doc conversion) for the easy ~70%, **in-account**.
2. **Hosted, isolated service** (Muneem pattern) for heavy/ML extraction, storage, and analytics ? fed only the minimized, non-PII handoff.

## Decision log

| # | Decision | Rationale |
| --- | --- | --- |
| ADR-001 | Pipeline (ingest ? classify ? extract ? redact ? handoff), not a copy loop |
```

```
Treat mail as structured data; make extraction re-runnable over a durable record |
202     | ADR-002 | Extraction (this repo) is decoupled from processing (Muneem & siblings) |
Isolate domain logic + encrypted storage + analytics downstream; clean contract seam |
203     | ADR-003 | Storage via a `Store`/`Sink` interface ? Sheets v1, remote/Muneem later |
Extensible by design; never baked into the code |
204     | ADR-004 | Egress: non-PII only, to our own isolated hosted service, no third parties;
scrub + encrypt; minimize | Privacy north star; preserves trust; in-account baseline needs no
egress |
205     | ADR-005 | Audience: <100 (Testing) now, public later; CASA deferred | Harden + learn
value before investing in assessment/infra |
206     | ADR-006 | First verticals: grocery receipts + bank statements (? Muneem) | Volume +
the high-fidelity handoff path |
207     | ADR-007 | Preserve current strengths (pure/shell, quotas, triggers, per-user,
versioning/provenance) | The recent rewrite is exactly why this pivot is feasible |
208     | ADR-008 | **Option A:** this repo is the sole Gmail-touching component ? in-account,
auditable, self-deployable; Muneem orchestrates but never holds the Gmail grant. Codified as the
binding [auditability invariant](#auditability-invariant-binding). | Verifiable privacy is the
product's moat and shrinks future verification scope; it concentrates the trust-critical surface
in one open, in-account place. |
209
210     ## Open items for the next iteration
211
212     - **Muneem handoff contract:** schema + transport (e.g. `UrlFetch` to a signed
213       endpoint vs. write-to-a-shared-store) ? define jointly with `avidullu/muneem`.
214     - **Per-vertical "non-PII" definition** for receipts and bank statements (what is
215       sent vs. tokenized vs. kept in-account).
216     - **First extractor slice** to build in Phase 1 (recommend a narrow
217       "receipt-smelling" detector) and its golden-email test fixtures.
218
```

4. user (2026-06-10T16:13:34.949Z)

```
1       # Roadmap
2
3       Next steps for the modernised project, grouped by effort and grounded in the
4       current Apps Script / Gmail platform constraints.
5
6       ## Platform constraints that shape the design
7
8       For consumer (gmail.com) accounts:
9
10      - **6 minutes** max per execution; **90 minutes/day** total trigger runtime;
11        **20 triggers** per script per user; ~**20,000** Gmail read calls/day.
12        ([Apps Script quotas](https://developers.google.com/apps-
script/guides/services/quotas))
13      - There is **no clearly documented daily Drive file-create quota**, so
14        `MAX_FILES_ADD_PER_DAY` (240) is a self-imposed heuristic, not an API limit.
15      - Gmail's
[`users.history.list`](https://developers.google.com/workspace/gmail/api/guides/sync)
16        with a stored `startHistoryId` is the purpose-built incremental sync (~2 quota
17        units vs 5 for a list call) and returns only changes. A `historyId` can expire
18        (HTTP 404 ? fall back to full sync), so it complements the date-window
19        backfill rather than replacing it.
20      - clasp can deploy from CI using a `CLASPRC_JSON` secret (contents of
21        `~/.clasprc.json`); service accounts do **not** work with the Apps Script API,
22        so use a dedicated owner account's token.
23        ([clasp #225](https://github.com/google/clasp/issues/225))
24
25      ## Quick wins
26
27      - [x] **Cover the orchestration shells with tests.** `sync.ts` / `ui.ts` had no
28        coverage ? the trickiest logic (trigger fan-out, quota reset, catch-up loop,
29        `processInput` state reset). *(done in this branch; see `tests/sync.test.ts`,
30        `tests/ui.test.ts`)*
31      - [x] **Raise the per-run wall-clock budget.** Was capped at 1 minute against a
```

```
  6-minute ceiling, making backfill ~6× slower than necessary. Now
  `MAX_RUNTIME_MS` (5 min) in `src/config.ts`, threaded through `hasTime`.
- [x] **Guard against trigger leakage.** A `LockService` script lock prevents
  concurrent runs from stacking triggers, deletion is scoped to the sync
  handler, and a cap prunes any excess. *(done in this branch; `src/sync.ts`)*
- [x] **Add a Jest `coverageThreshold`** so CI fails if coverage regresses.
  *(done; `jest.config.js`, gated at 85% stmts/lines/funcs, 75% branches,
  `entry.ts` bundler glue excluded)*

## Medium

- [x] **CI/CD auto-deploy on merge to `master`.** A gated `deploy` job writes
  `CLASPRC_JSON` ? `~/.clasprc.json` and runs `clasp push` / `clasp deploy`
  against the committed `.clasp.json`. *(done; `.github/workflows/ci.yml`.
  Requires only the `CLASPRC_JSON` repo secret; skips cleanly if unset.)*
- [x] **Cross-run dedup by message/attachment ID.** A bounded FIFO of processed
  Gmail message ids (`src/processed.ts`, capped at `MAX_PROCESSED_IDS`) is
  persisted in user properties; `updateDrive` skips messages already in the set
  and reports newly-completed ids back to `sync.ts`. A message is only marked
  done when all its attachments were copied cleanly (not truncated by the daily
  cap or a transient error), so re-scanned windows skip the Drive
  `getFilesByName` probe. *(done; `src/processed.ts`, `src/drive.ts`,
  `src/sync.ts`, `src/state.ts`)*
- [x] **Observability.** Level-prefixed structured logging (`src/logger.ts`),
  per-day run stats accumulator (`src/stats.ts` ? runs/files/errors/last error,
  persisted via `PROP_RUN_STATS`), and an opt-in daily summary
  (`src/notify.ts`) logged on each new-day rollover. The summary email is OFF by
  default; enabling it needs `ENABLE_SUMMARY_EMAIL` + the `script.send_mail`
  scope. *(done)*

## Larger / architectural

- [x] **Hybrid incremental sync.** Keep the date-window scan for the initial
  backfill; once caught up (`isToday`), switch the steady state to the Gmail
  advanced service + `history.list`/`startHistoryId`, storing `historyId` in
  properties and falling back to full sync on 404. *(done; pure logic in
  `src/historySync.ts`, advanced-service shell in `src/gmailHistory.ts`, wired
  into `src/sync.ts`. Gated behind `ENABLE_INCREMENTAL_SYNC` (OFF by default)
  for gradual rollout over the proven backfill. The Gmail advanced service is
  declared in `appsscript.json`; no new OAuth scope ? `gmail.readonly` covers
  history reads. Backfill captures the mailbox `historyId` before processing so
  mail arriving mid-run is still caught; expiry (HTTP 404) clears the cursor and
  rewinds to a bounded date-window re-scan.)*
- [ ] **More structure options & Shared Drive support** ? by-sender, nested
  `Year/Month`, label-driven folders; currently My Drive only.

## Open-source & auditability readiness

Concrete prep so the [auditability invariant](docs/ARCHITECTURE.md#auditability-
invariant-binding)
is real, not aspirational, before/when the repo is opened up.

- [x] Root `LICENSE` file (MIT, matching the source headers).
- [x] Egress guard ? a test that fails CI if `src/` makes any network call
  (`tests/auditability.test.ts`). Update it deliberately when the user-endpoint
  egress lands (assert the allowlisted destination, don't remove the guard).
- [ ] `SECURITY.md` ? how to report issues + the current data-flow guarantees
  (today: reads Gmail, writes the user's own Drive, **no egress**).
- [ ] **"Deploy your own from source"** guide ? self-host the GAS project end to
  end, so the trust claim is "run the code you read."
- [ ] **Reproducible / verifiable build** ? pin the toolchain and document how to
  diff a deployed `dist/Code.js` against a build of a given commit.
- [ ] `CONTRIBUTING.md` carrying the auditability PR checklist.
```

5. user (2026-06-10T16:13:35.128Z)

```
1       # Gmail Attachments ? Google Drive
2
3       A Google Apps Script that copies your Gmail attachments into a `GmailAttachments`
4       folder in Google Drive and keeps it updated as new mail arrives. You choose how
5       the attachments are organised (by content type, by month, or by year) and how
6       far back the first sync should reach.
7
8       Originally a 2013 Chrome Web Store app; this is a modernised rebuild on the
9       **V8 runtime** with a TypeScript source, an esbuild build, and a local Jest test
10      suite. The web app runs **as each accessing user** (`executeAs: USER_ACCESSING`),
11      so it can serve just you, a small group, or a Google Workspace org ? each
12      person's attachments are copied within their own account. See
13      [Multi-user deployment / publishing](#multi-user-deployment--publishing).
14
15      > **Where this is headed:** attachment archiving is the hardened core of a larger
16      > goal ? turning email into a structured, queryable record of one's life
17      > (receipts, statements, travel, ?) via an ingest ? classify ? extract ? handoff
18      > pipeline, with domain processing isolated in downstream services. See
19      > [`docs/VISION.md`](./docs/VISION.md) and
[`docs/ARCHITECTURE.md`](./docs/ARCHITECTURE.md).
20
21      ## How it works
22
23      - `doGet` serves a setup form (`html/index.html`).
24      - `processInput` stores your choices and schedules the first background run.
25      - `syncUserDrives` is a self-rescheduling time-based trigger that walks Gmail in
26        30-day windows, copying allowed attachments into Drive while respecting a
27        daily file cap (`MAX_FILES_ADD_PER_DAY`) and a per-run wall-clock budget.
28
29      State lives in `PropertiesService` (user properties). Existing Drive files are
30      never modified ? re-running setup only resets the script's own bookkeeping.
31
32      ## Project layout
33
34      | Path | Purpose |
35      | --- | --- |
36      | `src/` | TypeScript source. Pure logic (`query`, `naming`, `quota`, `contentTypes`) is
separated from the Google-service shells (`drive`, `sync`, `ui`, `state`). |
37      | `src/entry.ts` | Bundle entry; esbuild exposes `doGet` / `processInput` /
`syncUserDrives` as globals. |
38      | `html/index.html` | The setup form (pushed to Apps Script as `index`). |
39      | `appsscript.json` | Manifest: V8 runtime, OAuth scopes, web-app settings. |
40      | `tests/` | Jest unit tests. |
41      | `dist/` | Build output that `clasp` pushes (git-ignored). |
42
43      ## Develop
44
45      ```bash
46      npm install
47      npm run typecheck   # tsc --noEmit
48      npm run lint        # eslint
49      npm test            # jest
50      npm run build       # bundle src -> dist/Code.js (+ manifest + html)
51      ```
52
53      ## Deploy (requires your Google account)
54
55      1. `npm install -g @google/clasp && clasp login`
56      2. The project is already linked via the committed `.clasp.json` (`scriptId` +
57         `rootDir: dist`). To target a different project, edit `scriptId`.
58      3. `npm run push` ? builds and `clasp push`es `dist/`.
59      4. In the Apps Script editor, deploy as a **Web app**. The manifest already sets
60         *Execute as: User accessing* and *Who has access: Anyone*, so each visitor
```

```
   runs against their own account. Open the URL and submit the form.
5. Set `timeZone` in `appsscript.json` to your own zone ? the Gmail
   `before:`/`after:` day boundaries follow it.

> Using it only for yourself? It still works as-is ? you're simply the only
> user. To let others in, follow the runbook below.

### Auto-deploy from CI

Pushing to `master` runs the `deploy` job in `.github/workflows/ci.yml`, which
`clasp push`es and `clasp deploy`s the build. It needs a single repository
secret:

- `CLASPRC_JSON` ? the full contents of `~/.clasprc.json` after `clasp login`
  (generate it from a dedicated owner account; the token refreshes over time).

The `scriptId` lives in the committed `.clasp.json` (it is not sensitive ? it
appears in the web-app URL and grants nothing without the OAuth token). If the
secret is unset the job logs a warning and skips, so forks/CI without
credentials stay green.

### OAuth scopes

`gmail.readonly` (we only read mail), `drive` (create/enumerate folders and
files), and `script.scriptapp` (manage the background triggers).

## Observability

Each run logs structured, level-prefixed lines (`[INFO]` / `[ERROR]` / `[DEBUG]`)
to the Apps Script execution log / Stackdriver, and accumulates per-day stats
(runs, files added, errors, last error). On the first run of a new calendar day
the previous day's summary is logged.

### Optional daily summary email

Off by default. To receive the daily summary by email:

1. Set `ENABLE_SUMMARY_EMAIL = true` (and optionally `SUMMARY_EMAIL_RECIPIENT`)
   in `src/config.ts`.
2. Add the mail-send scope to `appsscript.json` `oauthScopes`:
   `https://www.googleapis.com/auth/script.send_mail`
3. Rebuild and redeploy; you'll be prompted to re-consent to the new
   "send email as you" permission.

## Multi-user deployment / publishing

The app is already wired for multiple users (runs as the accessing user, with
all state in *their* `getUserProperties`, a per-user lock, and per-user
triggers). What it takes to actually let other people in depends on the audience
? and this project deliberately targets the **no-verification** zone: a small
named group and/or a single Workspace org. That avoids Google's brand /
restricted-scope verification **and** the annual paid third-party CASA security
assessment that a fully public launch with `gmail.readonly` + `drive` would
require.

### One-time Google Cloud setup

1. **Use a standard Cloud project** (the default Apps Script one can't host a
   managed consent screen). Create a project in the
   [Cloud Console](https://console.cloud.google.com/), then in the Apps Script
   editor: **Project Settings ? Google Cloud Platform (GCP) Project ? Change
   project** and paste the project number. *This switch is permanent.*
2. **Enable APIs** in that project: **Gmail API** (required by the advanced Gmail
   service this app uses) and **Google Drive API**.
3. **Configure the OAuth consent screen / Branding**: app name
```

```
126        (`Gmail Attachments to Drive`), a user-support email, a developer-contact
127        email, and ? recommended ? a homepage and a **privacy policy URL**. A ready
128        policy lives at [`PRIVACY.md`](./PRIVACY.md); host it (e.g. GitHub Pages) and
129        add that domain under *Authorized domains*.
130
131    ### Pick an audience (same build, different consent config)
132
133    | | Small group | Workspace org |
134    | --- | --- | --- |
135    | **User type** | External, status **Testing** | **Internal** |
136    | **Who can use it** | Up to **100** Google accounts you add as *Test users* | Everyone
in your Workspace org |
137    | **Consent screen** | Shows a "Google hasn't verified this app" warning (expected) | No
warning |
138    | **Caveat** | Test users re-consent ~weekly (7-day refresh-token expiry) | Org-only;
not usable by external gmail.com accounts |
139    | **Verification / CASA** | None | None |
140
141    Then deploy (the CI `clasp push`/`deploy` flow is unchanged) and share the
142    `/exec` web-app URL. **Note:** switching the GCP project / `executeAs` forces
143    everyone ? including you ? to re-authorize once.
144
145    ### Going fully public (out of scope here)
146
147    Letting *anyone* use it (External + Production) requires brand verification,
148    restricted-scope verification, domain verification, and an **annual CASA
149    security assessment** (~$1?5k+, 6?16 weeks). If you ever go there, first narrow
150    `drive` ? `drive.file` (the app only touches files it creates) to shrink the
151    review surface ? though `gmail.readonly` remains restricted regardless.
152
153    ## License
154
155    MIT ? see headers in source files.
156
```

6. user (2026-06-10T16:13:41.318Z)

```
1      # Privacy Policy ? Gmail Attachments to Drive
2
3      _Last updated: 2026-05-31_
4
5      **Gmail Attachments to Drive** ("the app") is a Google Apps Script web app that
6      copies attachments from your Gmail into a `GmailAttachments` folder in your own
7      Google Drive.
8
9      ## What the app accesses
10
11     When you authorize the app, it requests permission to:
12
13     - **Read your Gmail** (`gmail.readonly`) ? to find messages with attachments and
14       read those attachments so they can be copied.
15     - **Manage your Google Drive** (`drive`) ? to create the `GmailAttachments`
16       folder and its sub-folders and to write the copied attachment files.
17     - **Run in the background** (`script.scriptapp`) ? to schedule the periodic sync
18       on your behalf.
19
20     ## How your data is used
21
22     - The app runs **entirely within your own Google account**, as you
23       (the web app is deployed with `executeAs: USER_ACCESSING`).
24     - Attachments are copied **from your Gmail to your Drive only**. Your email
25       content and files are **never sent to, stored on, or transmitted through any
26       third-party server**. The author of the app has no access to your data.
27     - The only data the app stores is small bookkeeping state (your folder-structure
28       choice, a sync cursor, counters, and recently-processed message IDs) in
```

```
29        **Google's per-user `PropertiesService`**, inside your account.
30
31      ## Data sharing and retention
32
33      - **No data is shared** with the developer or any third party.
34      - Bookkeeping state lives in your account for as long as the app is installed.
35        Re-running setup clears it. Files already copied to your Drive are yours and
36        are never modified or deleted by the app.
37
38      ## Revoking access
39
40      You can revoke the app's access at any time at
41      [myaccount.google.com/permissions](https://myaccount.google.com/permissions).
42      This stops future syncs; attachments already copied to your Drive remain.
43
44      ## Contact
45
46      Questions or issues:
47      [github.com/avidullu/gmail-attachments-to-gdrive](https://github.com/avidullu/gmail-
attachments-to-gdrive)
48      (open an issue).
49
```

7. user (2026-06-10T16:13:42.264Z)

```
1       {
2         "timeZone": "America/Los_Angeles",
3         "runtimeVersion": "V8",
4         "exceptionLogging": "STACKDRIVER",
5         "dependencies": {
6           "enabledAdvancedServices": [
7             {
8               "userSymbol": "Gmail",
9               "serviceId": "gmail",
10              "version": "v1"
11            }
12          ]
13        },
14        "oauthScopes": [
15          "https://www.googleapis.com/auth/gmail.readonly",
16          "https://www.googleapis.com/auth/drive",
17          "https://www.googleapis.com/auth/script.scriptapp"
18        ],
19        "webapp": {
20          "access": "ANYONE",
21          "executeAs": "USER_ACCESSING"
22        }
23      }
24
```

8. user (2026-06-10T16:13:42.731Z)

```
1       {
2         "name": "gmail-attachments-to-gdrive",
3         "version": "2.0.0",
4         "description": "Google Apps Script that syncs Gmail attachments into Google Drive.",
5         "license": "MIT",
6         "private": true,
7         "scripts": {
8           "build": "node esbuild.config.mjs",
9           "typecheck": "tsc --noEmit",
10          "lint": "eslint src tests",
11          "test": "jest",
12          "test:coverage": "jest --coverage",
13          "push": "npm run build && clasp push",
```

```
    "deploy": "npm run build && clasp deploy"
  },
  "devDependencies": {
    "@google/clasp": "^2.4.2",
    "@types/google-apps-script": "^1.0.83",
    "@types/jest": "^29.5.12",
    "@types/node": "^20.12.7",
    "esbuild": "^0.21.5",
    "eslint": "^9.5.0",
    "jest": "^29.7.0",
    "ts-jest": "^29.1.4",
    "typescript": "^5.4.5",
    "typescript-eslint": "^8.0.0"
  }
}

```

9. user (2026-06-10T16:13:43.041Z)

```
MIT License

Copyright (c) 2013 Avi Dullu

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in
all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
THE SOFTWARE.

```

10. user (2026-06-10T16:13:49.804Z)

```
No files found
```

11. user (2026-06-10T16:13:50.097Z)

```
No files found
```

12. user (2026-06-10T16:13:50.456Z)

```
import { readFileSync, readdirSync } from "fs";
import { join } from "path";

/*
 * Enforces the auditability invariant (docs/ARCHITECTURE.md): this repo is the
 * only code that touches a user's Gmail, and it makes NO network egress today.
 *
 * The ONLY sanctioned way data may ever leave a user's account is a deliberate,
 * reviewed REDACT/MINIMIZE -> user's-own-endpoint path. When that lands, update
 * this guard to assert the single allowlisted destination ? do not delete it.
 */

```

```
13      // Apps Script's only outbound-network primitive.
14      const FORBIDDEN = /UrlFetchApp/;
15
16      function tsFiles(dir: string): string[] {
17        return readdirSync(dir, { withFileTypes: true }).flatMap((entry) => {
18          const full = join(dir, entry.name);
19          if (entry.isDirectory()) return tsFiles(full);
20          return entry.name.endsWith(".ts") ? [full] : [];
21        });
22      }
23
24      describe("auditability invariant", () => {
25        it("src/ makes no outbound network calls (no UrlFetchApp)", () => {
26          const offenders = tsFiles("src").filter((file) =>
27            FORBIDDEN.test(readFileSync(file, "utf8")),
28          );
29          expect(offenders).toEqual([]);
30        });
31      });
32
```

13. user (2026-06-10T16:13:58.197Z)

```
types
config.ts
contentTypes.ts
drive.ts
entry.ts
gmailHistory.ts
historySync.ts
logger.ts
naming.ts
notify.ts
processed.ts
query.ts
quota.ts
state.ts
stats.ts
sync.ts
ui.ts
types\gmailAdvanced.d.ts
auditability.test.ts
contentTypes.test.ts
drive.test.ts
gmailHistory.test.ts
historySync.test.ts
naming.test.ts
notify.test.ts
processed.test.ts
query.test.ts
quota.test.ts
setup.ts
stats.test.ts
sync.test.ts
syncIncremental.test.ts
ui.test.ts
ARCHITECTURE.md
VISION.md
```

14. user (2026-06-10T16:14:07.085Z)

```
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\notify.ts:37:
MailApp.sendEmail(to, "Gmail?Drive sync summary " + stats.day, summary);
```

15. user (2026-06-10T16:14:08.874Z)

```
No matches found
```

16. user (2026-06-10T16:14:09.033Z)

```
1       import {
2         ENABLE_SUMMARY_EMAIL,
3         SUMMARY_EMAIL_RECIPIENT,
4       } from "./config";
5       import { logError, logInfo } from "./logger";
6       import { dayKey, formatSummary } from "./stats";
7       import * as state from "./state";
8
9       /**
10       * Summarises the previous day's activity the first time a sync runs on a new
11       * calendar day. Always logs the summary; emailing is opt-in
12       * (ENABLE_SUMMARY_EMAIL, which requires the script.send_mail scope).
13       *
14       * `now` is injected for testability.
15       */
16      export function maybeSendDailySummary(now: Date): void {
17        const today = dayKey(now);
18        if (state.getSummaryLastSent() === today) {
19          return; // already summarised since the last day rolled over
20        }
21
22        // Summarise whatever the persisted accumulator holds (the prior day's run).
23        const stats = state.getRunStatsRaw();
24        if (!stats || stats.day === today) {
25          // Nothing from a prior day to report yet; arm for the next rollover.
26          state.setSummaryLastSent(today);
27          return;
28        }
29
30        const summary = formatSummary(stats);
31        logInfo(summary);
32
33        if (ENABLE_SUMMARY_EMAIL) {
34          try {
35            const to = SUMMARY_EMAIL_RECIPIENT || Session.getEffectiveUser().getEmail();
36            if (to) {
37              MailApp.sendEmail(to, "Gmail?Drive sync summary " + stats.day, summary);
38            }
39          } catch (e) {
40            logError("Failed to send summary email: " + e);
41          }
42        }
43
44        state.setSummaryLastSent(today);
45      }
46
```

17. user (2026-06-10T16:14:17.491Z)

```
48:**Goal:** pull messages and attachments from Gmail and Outlook into a local staging area.
50:- **Gmail:** [Gmail API](https://developers.google.com/gmail/api) over OAuth2. Read-only
scope (`gmail.readonly`). Supports server-side search queries (`has:attachment filename:pdf
from:hdfc newer_than:90d`) which is far cheaper than pulling everything and filtering locally.
Has good rate limits for personal use.
52:- **Why not IMAP:** worse search, no labels, harder OAuth, and we lose Gmail's powerful query
syntax. Use the native APIs.
54:Mechanically straightforward. The main design questions are: **how is "financial document"
detected** (sender allowlist? subject patterns? attachment heuristics? hybrid?), and **how is
incremental sync state tracked** (Gmail `historyId` / Graph `deltaLink`).
```

```
148:> **Gmail ? one specific bank's monthly statements ? unlocked ? parsed into `transactions` ?
stored in SQLite ? a CLI command that lists last month's transactions.**
184:§ 7 proposes "Gmail + one bank ? CLI". Confirm with user before building.
186:> **Resolved (2026-05-30): Build both tracks in parallel** ? the promo-corpus
parse?store?query spine *and* the Gmail ingestion plumbing, as two decoupled tracks that
converge at the first bank slice; first bank = **HDFC**. This refines § 7 (which proposed a
single end-to-end bank slice first). See [`docs/ROADMAP.md`](./docs/ROADMAP.md) §§ 1 and 4.
234:[Omitted long matching line]
```

18. user (2026-06-10T16:14:21.640Z)

```
No matches found
```

19. user (2026-06-10T16:14:29.981Z)

```
15:> **Track B:** Gmail OAuth2 + search + attachment download ?; the `muneem fetch` CLI (B.4) is
not
115:  parse ? store ? query           fetch ? stage ? catalog          fully decoupled)
131:Tracks A and B are genuinely independent: A needs no OAuth and no secrets; B needs no
parsing. As a single developer you can interleave them in any order ? neither blocks the other.
**Both must land before Milestone C**, where a fetched-and-unlocked HDFC PDF flows through the
parser for the first time.
157:### Track B ? Gmail ingestion · PARTIAL (OAuth + search + download ?; `muneem fetch` B.4
pending)
163:- [ ] **B.3 (S)** Incremental sync: persist Gmail `historyId`; on re-run, fetch only what's
new. Idempotent (re-running never duplicates `documents`, keyed by message id + attachment
hash).
164:- [ ] **B.4 (S)** Catalog fetched attachments as `documents` rows (status
`locked`/`unparsed`), with sha256 + staging path. `muneem fetch --since 90d` is the entry point.
165:- **Exit:** `muneem fetch` populates the staging dir + `documents` rows from a narrow real-
Gmail query. (Manual/local verification ? not CI; it touches a real account.)
169:> A fetched HDFC statement gets unlocked and parsed into real transactions. This is the
moment every stage runs once on real data the user cares about.
306:The DESIGN.md § 7 slice, hardened: **`muneem fetch` pulls HDFC statements from Gmail ?
unlocks them with the hybrid password strategy ? the HDFC parser writes normalized
`transactions` to SQLite ? `muneem txns list` shows last month's activity ? all locally, with
redacted logs, idempotent re-runs, green CI on synthetic fixtures, and a written evaluation of
where to go next.** The promo "forgotten deals" feature ships alongside as the Track A
deliverable.
```

20. user (2026-06-10T16:14:53.923Z)

```
4:import { logDebug, logError } from "./logger";
38:      logDebug("Found existing folder " + name);
42:  logDebug("Creating new folder " + name);
96:  logDebug("Already present folders: " + existingFolders.size);
112:        logDebug("Skipping already-processed message " + messageId);
129:          logDebug("Rejected file of type: " + contentType);
142:          logError("Could not resolve directory; using top-level folder.");
166:          logError("Attachment copy failed (message " + messageId + "): " + e);
```

21. user (2026-06-10T16:14:54.166Z)

```
1       import { ENABLE_DEBUGGING } from "./config";
2
3       /**
4        * Minimal structured logger. Each line is prefixed with a level so the Apps
5        * Script execution log / Stackdriver can be filtered (e.g. `severity:ERROR`).
6        * Kept dependency-free and side-effect-only so callers stay simple.
7        */
8       function emit(level: "DEBUG" | "INFO" | "ERROR", data: unknown): void {
9         Logger.log("[" + level + "] " + String(data));
10      }
11
```

```
12      /** Verbose log, only emitted when ENABLE_DEBUGGING is on. */
13      export function logDebug(data: unknown): void {
14        if (ENABLE_DEBUGGING) {
15          emit("DEBUG", data);
16        }
17      }
18
19      /** Informational log; always emitted. */
20      export function logInfo(data: unknown): void {
21        emit("INFO", data);
22      }
23
24      /** Error log; always emitted and surfaces in Stackdriver as an error. */
25      export function logError(data: unknown): void {
26        emit("ERROR", data);
27      }
28
```

22. user (2026-06-10T16:18:27.117Z)

```
Structured output provided successfully
```

23. assistant (2026-06-10T16:18:33.098Z)

```
Audit complete. The structured report has been returned via StructuredOutput.
```